0 1 0 1

Array e base 2

Tante caselle con un solo nome: la conversione in base 2 e i limiti dell'array

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027

Sapienza
Università di Roma

Convertire in base 2: i resti

Si divide per 2 finché il quoziente è 0: i resti, letti dall'ultimo al primo, sono le cifre

n	n / 2	resto
13	6	1
6	3	0
3	1	1
1	0	1



si legge dal basso verso l'alto

1 1 0 1

13 in base 2

Controllo

1101 = 8 + 4 + 0 + 1 = 13

Il primo resto

è la cifra più a destra (peso 1)

Un primo programma, che sbaglia

binaryErr.c: ogni resto viene stampato appena calcolato

binaryErr.c

```c
#include <stdio.h>

int main() {
    int n, resto;
    printf("numero intero positivo: ");
    scanf("%d", &n);
    while (n > 0) {
        resto = n % 2;
        printf("%d", resto);    // stampo subito il resto
        n = n / 2;
    }
    printf("\n");
}
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe binaryErr.c
$ ./app.exe
numero intero positivo: 13
1011
```

1011 invece di 1101

le cifre escono in ordine inverso

Il primo resto esce per primo

ma è la cifra più a destra: va stampato per ultimo

Serve ricordare i resti

per poi stamparli all'indietro

Un array: tante caselle con un solo nome

int cifra[8]: otto interi uno dopo l'altro, numerati da 0 a 7

cifra

1	0	1	1	0	0	0	0
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

dichiarare e riempire

```
int cifra[8];
cifra[0] = 1;        // la prima casella
cifra[3] = 1;
```

Indice

il numero della casella, tra parentesi quadre; si parte da 0

Un nome, un tipo

tutte le caselle sono int

L'ultima casella

ha indice 7, non 8

Dichiarare e inizializzare

La lunghezza è un numero fisso scritto nella dichiarazione

dichiarazioni

```
#define NMAX 100

int cifra[8];              // 8 caselle, valori non definiti
int c[4] = {3, 5, 7, 9};   // valori iniziali
double x[NMAX];            // 100 reali
int zero[NMAX] = {0};     // tutte a zero
x[99] = 0.23;              // l'ultima casella
```

Indici da 0 a n - 1

x[100] non esiste: nessuna casella

{0}

mette a zero tutte le caselle

#define NMAX

una lunghezza che si cambia in un punto solo

sizeof, misurato sul Mac

```
int: 4 byte, cifra[8]: 32 byte
double: 8 byte, x[100]: 800 byte
```

La base 2 con l'array

binary.c: i resti vanno nelle caselle, poi si stampano dall'ultima alla prima

binary.c (dopo l'include)

```c
#define NMAX 32

int main() {
    int n, i, ncifre = 0;
    int cifra[NMAX];              // le cifre binarie
    do {
        printf("numero intero positivo: ");
        scanf("%d", &n);
    } while (n <= 0);
    while (n > 0) {
        cifra[ncifre] = n % 2;    // il resto va nella casella ncifre
        n = n / 2;
        ncifre++;
    }
    for (i = ncifre - 1; i >= 0; i--) {    // dall'ultima alla prima
        printf("%d", cifra[i]);
    }
    printf("\n");
}
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe binary.c
$ ./app.exe
numero intero positivo: 13
1101
$ ./app.exe
numero intero positivo: 100
1100100
```

ncifre	n	cifra[ ]	n / 2
0	13	1	6
1	6	0	3
2	3	1	1
3	1	1	0

Attenzione ai limiti

a ha tre caselle, ma il ciclo ne legge cinque: il compilatore non dice nulla

fuori.c

```c
#include <stdio.h>

int main() {
    int a[3] = {10, 20, 30};
    int i;
    for (i = 0; i < 5; i++) {
        printf("a[%d] = %d\n", i, a[i]);
    }
}
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe fuori.c
$ ./app.exe
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 2
a[4] = -2120810281
```

a[3] e a[4]

non esistono: si leggono altre zone di memoria

Nessun errore

il programma compila e gira: comportamento indefinito

Valori diversi

su ogni computer; in scrittura può bloccarsi

Esercizi

Da fare a casa, poi si controlla con un programma in C

1

Base 8

Il programma legge un numero intero positivo e lo stampa in base 8, come binary.c fa per la base 2: si divide ripetutamente per 8, si salvano i resti in un array e poi si stampano le cifre all'indietro.

per controllare:

```
100: 144
511: 777
8: 10
```

2

A rovescio

Dichiara un array di 7 interi inizializzato con i valori 4 9 2 7 1 8 5. Stampa su una riga gli elementi nell'ordine inverso, dall'ultimo al primo. Poi stampa la somma degli elementi che stanno nelle caselle di indice pari (0, 2, 4 e 6).

per controllare:

```
5 8 1 7 2 9 4
somma indici pari: 12
```

3

Complemento

Leggi un intero positivo in base 10 e convertilo in base 2, salvando le cifre in un array come in binary.c. Il complemento si ottiene scambiando le cifre: ogni 0 diventa 1 e ogni 1 diventa 0. Stampa il numero in binario e, sulla riga sotto, il suo complemento con lo stesso numero di cifre.

per controllare:

```
37:
100101
011010
```

Compila con `gcc -Wall -Wextra -o app.exe nome.c`

Riepilogo

1	**Array**	int a[8]: 8 caselle con lo stesso nome, indici da 0 a 7; {0} per azzerare; la lunghezza è fissa (#define NMAX)
2	**Ciclo e indice**	un for con indice i per scorrere le caselle; fuori dai limiti il compilatore non dice nulla
3	**Un uso**	i resti della base 2 da stampare all'indietro, con le cifre in un array (stesso schema per la base 8)

Extra: i numeri primi

Un numero è primo se non ha divisori tra 2 e la sua radice quadrata; 1 non è primo

primo.c (dopo gli include)

```c
int main() {
    int n, j, jmax;
    int primo = 1;                  // 1 = vero
    do {
        printf("numero intero (almeno 2): ");
        scanf("%d", &n);
    } while (n < 2);
    jmax = (int) sqrt(n);           // fino a sqrt(n)
    for (j = 2; j <= jmax; j++) {
        if (n % j == 0) {
            primo = 0;
            break;                  // esce dal ciclo
        }
    }
    if (primo) {
        printf("%d primo\n", n);
    } else {
        printf("%d non primo (divisibile per %d)\n", n, j);
    }
}
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe primo.c -lm
$ ./app.exe
numero intero (almeno 2): 17
17 primo
$ ./app.exe
numero intero (almeno 2): 91
91 non primo (divisibile per 7)
$ ./app.exe
numero intero (almeno 2): 2
2 primo
```

break

esce dal ciclo al primo divisore

n % j == 0

j divide n senza resto